

MLDA@EEE 2026

Espressif Industry Projects

Low-Power Voice Enhancement Model

Background

Voice enhancement technology is critical for improving audio quality in noisy environments, enabling clear voice capture for applications like voice assistants, conferencing systems, and IoT devices. Traditional voice processing algorithms often struggle with real-time performance and high computational costs, especially on resource-constrained devices. Recent advancements in lightweight neural networks, such as **GTCRN (Grouped Temporal Convolutional Recurrent Network)**, have made it possible to achieve high-quality voice enhancement with minimal power consumption.

Model Deliverables

Lightweight Voice Enhancement Model

- A trained neural network capable of real-time denoising and speech enhancement.
- Target metrics: <50 ms latency, optimized for ESP32-S3 or ESP32-P4 class devices.
- Demonstrated robustness in multiple noisy environments (industrial, outdoor, etc.).

Model Benchmark Report

- Model size, FLOPs, memory footprint, and latency measurements on target hardware.
- Comparison against baseline models (e.g., GTCRN, NSNET2).
- Quantized Model Variant (INT8 / mixed precision)
- Optimized version for inference on embedded hardware (TFLite Micro or ESP-DL).

Firmware Deliverables

ESP-IDF-based Firmware

- Integrates the trained model into an embedded inference pipeline.
- Note that ESP-IDF base factory firmware is open source and can run object detection models.
- Optional Web UI or UART-based control for testing enhancement levels.

Model Inference Demo

- Real-time demo: noisy voice in → enhanced voice out.
- Possibly includes an A/B switch or recording comparison for evaluation.

Data Collection & Preprocessing Scripts

- Python tools for recording, mixing, and augmenting noisy speech datasets.
- Dataset documentation (noise sources, SNR levels, etc.).

Evaluation Deliverables

Performance Metrics

- Objective: PESQ, STOI, SDR, or SNR improvement.
- Subjective: Human listening test results.

Power Profiling Report

- Measurement of CPU utilization, inference time, and current draw.

Documentation Deliverables

Technical Report (Final Submission)

- System architecture, model design, optimization methods, testing results.
- Deployment challenges and future improvements.

Presentation Deck

- Summary of workflow, results, and live demo for final evaluation day.
-

Low-Power Pedestrian Detection Model

Background:

Pedestrian detection is a key component of advanced driver-assistance systems (ADAS), autonomous vehicles, and smart surveillance systems. However, traditional detection methods often face challenges such as high computational complexity, limited accuracy in crowded environments, and significant power consumption, which hinders deployment on edge devices. Recent research has focused on lightweight deep learning models like **LP-YOLO**, which reduce computational costs while maintaining high accuracy.

Model Deliverables

Lightweight Pedestrian Detection Model

- A deep learning model (inspired by LP-YOLO or YOLO-Nano) capable of detecting pedestrians in real-time on low-power embedded devices (ESP32-S3 / ESP32-P4).
- Target metrics:
 - Inference latency <100 ms per frame
 - Power consumption within device limits (<300 mW typical)
 - High detection accuracy (mAP > 0.75 on test set)

Quantized Model for Edge Inference

- INT8 / mixed-precision optimized version for deployment using ESP-DL or TFLite Micro.
- Benchmarked for accuracy vs performance trade-offs.

Model Comparison Report

- Comparison between baseline models (LP-YOLO, Tiny-YOLO, ESP-DL sample networks).
- Documentation of model size, parameters, FLOPs, and latency.

Firmware / Software Deliverables

ESP-IDF Based Application

- Integrates the trained model into an embedded inference pipeline.
- Visualization via onboard LCD / web dashboard (bounding boxes or detection count overlay).
- Note that ESP-IDF base factory firmware is open source and can run object detection models.

Inference Pipeline Integration

- Preprocessing: Image capture → resize → normalization
- Inference: Model runs via ESP-DL / TFLite Micro
- Post-processing: Non-max suppression, bounding box display

Dataset Handling Tools

- Python scripts for dataset preprocessing (image resizing, annotation format conversion).
- Optional augmentation pipeline for improving robustness (lighting, occlusion, blur).

Evaluation Deliverables

Performance & Accuracy Metrics

- Mean Average Precision (mAP), Precision-Recall curve, FPS, and power consumption.
- Confusion matrix and error analysis.

Power & Efficiency Profiling

- Measurement of inference latency, CPU load, and power draw (via INA219 or USB power meter).

Comparative Analysis Report

- Trade-off study: accuracy vs energy vs latency.

Documentation Deliverables

Technical Report (Final Submission)

- System overview, model architecture, optimization techniques, evaluation methodology, results, and discussion.

Presentation Slides

- Visual summary for final showcase or industry review, including demo setup and results.

Demonstration Video

- 1–2 minute video showing detection running in real-time on an ESP device.

For any Questions, please reach out to (via telegram):

Unnav Sharma (VP, Business Development) at @Unnav26

Jing Chen (Director, Business Development) at @jingchen2004

Sign Up Form: <https://forms.gle/M6FgEnLRfm6WWbx57>